

```
import numpy as np

# To print numpy scalars as '3.0' rather than
'np.float64(3.0)'
np.set_printoptions(legacy="1.21")

def shortestPath(orig_node, df_nodes, df_links,
forward_star):
    """
        Find the shortest paths from orig_node to all
other nodes using Dijkstra's algorithm.

        Parameters
        -----
        orig_node      : int          â€” the origin node ID
        df_nodes       : DataFrame    â€” node table (indexed
by 'node')
        df_links       : DataFrame    â€” link table with
'cost' column (indexed by (init_node, term_node))
        forward_star  : dict          â€” maps each node to
its list of directly reachable downstream nodes

        Returns
        -----
        L              : dict â€” shortest path cost from
orig_node to each node
        q              : dict â€” backnode of each node on the
shortest path (-1 if not set)
        odpath : dict â€” key: (orig_node, dest_node),
value: list of node IDs on the shortest path
        """

    # Initialise the label (cost) and backnode
dictionaries
    L = {} # Label dictionary, shortest path cost
from origin to each node
    q = {} # Backnode dictionary, preceding node on
the current shortest path
    Q = [] # Queue to store all unvisited nodes

    # Loop through all nodes in df_nodes to
initialise L, q, and Q
    for row in df_nodes.itertuples(index=False):
        L[row.node] = np.inf # Set the initial cost
for all nodes to infinity
        q[row.node] = -1 # Initialise the backnode
as -1 (no backnode initially)
        Q.append(row.node) # Add the node to the
unvisited queue

    L[orig_node] = 0.0 # Set the shortest path cost
of the origin node to 0

    # Main loop for Dijkstra's algorithm
    while len(Q) > 0: # Continue until all nodes
have been visited
        i = -1 # Variable to hold the current node
        minL = np.inf # Initialize the minimum cost
as infinity

        # Find the node with the smallest shortest
path cost
        # in the unvisited queue Q
        for u in Q:
            if L[u] <= minL: # If the cost of u is
less than the current minimum
                minL = L[u] # Update the minimum
cost

                i = u # Set the current node to u

        Q.remove(i) # Remove the selected node i
from the unvisited queue

        # Update L and q for all downstream nodes of
the current node i
        # by looping through the forward star of node
i
        for j in forward_star[i]:
            if j in Q: # If the downstream node j is
still in the unvisited queue
                # Get the link travel time for
traveling from node i to j
                t_ij = df_links.at[(i, j), "cost"]

                # Calculate the new shortest path
cost to reach node j via node i
                newL = L[i] + t_ij
                if newL < L[j]: # If the new cost is
smaller than the current cost
                    L[j] = newL # Update the
shortest path cost for node j
                    q[j] = i # Set the backnode of
node j to be node i

        # Create a odpath dictionary to store the
shortest paths
        # between the origin and all other nodes
(destinations)
        odpath = {} # key:(orig,dest), value:list of
nodes in the shortest path

        # Reconstruct the shortest path for each
destination node
        for row in df_nodes.itertuples(index=False):
            dest_node = row.node

            # Check if the destination node is reachable
            if dest_node == orig_node or L[dest_node] ==
np.inf:
                continue

            path = [dest_node] # Start with the
destination node
            current = dest_node # Set the current node
as the destination node

            # Trace back the path from the destination to
the origin using the backnode dictionary q
            while q[current] != -1: # While there is a
backnode to trace back
                # Insert the backnode of the current node
at the start of the path
                path.insert(0, q[current])
                current = q[current] # Move to the
previous node

            # Store the path in the odpaths dictionary
            odpath[(orig_node, dest_node)] = path

        return L, q, odpath

def updateLinkCost(df_links):
    """
        Update link costs (travel times) in-place using
the BPR function.

        Parameters
        -----
        df_links : DataFrame â€” link table with 'flow',
'free_flow_time', 'b',
                                'capacity', 'power'
columns; updates 'cost' in-place.

        Input:  df_links[['flow', 'capacity',
'free_flow_time', 'b', 'power']]
        Output: df_links['cost'] (in-place)
        """
        for link in df_links.itertuples(index=True):
            t = link.free_flow_time * (
                1 + link.b * pow(link.flow /
link.capacity, link.power)
            )
            df_links.at[link.Index, "cost"] = t

def updateOdShortestPathTime(df_nodes, df_links,
df_ods, forward_star):
    """
        Update shortest path travel times and paths for
all OD pairs in df_ods.

        Iterates only over origins and destinations that
appear in df_ods
        (not all nodes), then calls shortestPath() and
updates df_ods in-place.

        Parameters
        -----
        df_nodes      : DataFrame â€” node table (indexed
by 'node')
        df_links       : DataFrame â€” link table; uses
'cost' column
        df_ods         : DataFrame â€” OD table; updates
'shortest_path_time' and
                                'shortest_path' in-
place
        forward_star  : dict          â€” maps each node to
its list of downstream nodes

        Input:  df_links['cost'], df_ods['demand']
        Output: df_ods[['shortest_path_time',
'shortest_path']] (in-place)
        """
        # Loop over unique origin nodes that appear in
df_ods.
        for orig_node in df_ods["orig"].unique():
            L, q, odpath = shortestPath(orig_node,
df_nodes, df_links, forward_star)

            # Loop over destination nodes for this
origin.
            for dest_node in df_ods[df_ods["orig"] ==
orig_node]["dest"]:
                od_index = (orig_node, dest_node)
                # Ensure that od_index is in odpath
before accessing it to avoid KeyError
                if od_index in odpath:
                    df_ods.at[od_index, "shortest_path"]
= odpath[od_index]
                    df_ods.at[od_index,
"shortest_path_time"] = L[dest_node]

def allOrNothingAssignment(df_nodes, df_links,
df_ods, forward_star):
    """
        Perform All-or-Nothing (AON) assignment.

        Resets df_links['aon_flow'] to zero, then for
each OD pair calculate the shortest path and
loads the full OD demand onto every link along
the shortest path.

        Parameters
        -----
        df_nodes      : DataFrame â€” node table
        df_links       : DataFrame â€” link table; uses
'cost', updates 'aon_flow' in-place
        df_ods         : DataFrame â€” OD table; uses
'demand'
        forward_star  : dict          â€” maps each node to
its list of downstream nodes

        Input:  df_links['cost'], df_ods['demand']
        Output: df_links['aon_flow'] (in-place)
        """
        # Reset All-or-Nothing flow values to zeros
        df_links["aon_flow"] = 0.0

        # Loop over unique origin nodes present in df_ods
        for orig_node in df_ods["orig"].unique():
            # Find shortest paths from orig node to all
other nodes based on the current link costs
            L, q, odpath = shortestPath(orig_node,
df_nodes, df_links, forward_star)

            # Loop over destination nodes for this origin
            for dest_node in df_ods[df_ods["orig"] ==
orig_node]["dest"]:
                od_index = (orig_node, dest_node)

                # Check if a shortest path exists for
this OD pair
                if od_index in odpath:
                    path = odpath[od_index] # Retrieve
the shortest path

                    # Assign all demand to the links
along the shortest path
                    path_flow = df_ods.at[od_index,
"demand"]
                    for i in range(len(path) - 1):
                        link_index = (path[i], path[i +
1])
                        df_links.at[link_index,
"aon_flow"] += path_flow

def calculateConvergenceMeasures(df_links, df_ods):
    """
        Calculate the Relative Gap (RG) and Average
Excess Cost (AEC).

        Parameters
        -----
        df_links : DataFrame â€” link table; uses 'flow'
and 'cost'
        df_ods   : DataFrame â€” OD table; uses 'demand'
and 'shortest_path_time'

        Input:  df_links[['flow', 'cost']],
df_ods[['demand', 'shortest_path_time']]
        Output: relative_gap (float), avg_excess_cost
(float)
        """
        tx_sum = 0.0 # sum(t_ij * x_ij) -- total system
travel cost
        for link in df_links.itertuples(index=False):
            tx_sum += link.cost * link.flow

        kd_sum = 0.0 # sum(kappa^rs * d^rs) -- minimum
possible system travel cost
        d_sum = 0.0 # sum(d^rs) -- total demand
        for od in df_ods.itertuples(index=False):
            if od.demand > 0:
                d_sum += od.demand
                kd_sum += od.demand *
od.shortest_path_time

        relative_gap = tx_sum / kd_sum - 1 # (t.x) /
(kappa.d) - 1
        avg_excess_cost = (tx_sum - kd_sum) / d_sum #
(t.x - kappa.d) / d.l

        return relative_gap, avg_excess_cost

def shiftLinkFlows(step_size, df_links):
    """
        Update link flows using the update rule: x <-
lambda * x_star + (1 - lambda) * x

        Parameters
        -----
        step_size : float          â€” the step size lambda
        df_links  : DataFrame    â€” link table; reads
'aon_flow' (x*), updates 'flow' (x) in-place

        Input:  step_size, df_links[['flow', 'aon_flow']]
        Output: df_links['flow'] (in-place)
        """
        for link in df_links.itertuples(index=True):
            df_links.at[link.Index, "flow"] = (
                step_size * link.aon_flow + (1 -
step_size) * link.flow
            )
```